

Operating Systems - Chapter 27 Summary

Interlude: Thread API

- The POSIX Threads (pthreads) library provides functions to create, control, and synchronize threads.
- This chapter covers thread creation, completion, locks, and condition variables.

Thread Creation

- Threads are created with `pthread_create()`:
`int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);`
- Arguments: thread (thread ID), attr (attributes, usually NULL), start_routine (function pointer), and arg (argument to that function).
- The new thread begins executing in the start_routine function.
- Functions accept and return void pointers, allowing any type to be passed or returned.

Thread Completion

- `pthread_join(pthread_t thread, void **value_ptr)` waits for a thread to finish.
- value_ptr points to where the thread's return value will be stored.
- Passing NULL skips collecting a return value.
- WARNING: never return pointers to stack-allocated variables — they become invalid once the thread exits.

Locks (Mutexes)

- `pthread_mutex_t` provides mutual exclusion for critical sections.
- Functions: `pthread_mutex_lock()`, `pthread_mutex_unlock()`, `pthread_mutex_init()`, `pthread_mutex_destroy()`.
- Always initialize locks (PTHREAD_MUTEX_INITIALIZER or `pthread_mutex_init()`).
- Always check return codes; lock acquisition may fail.
- Trylock and timedlock variants exist but are rarely needed.

Condition Variables

- Used for signaling between threads (one thread waits for a condition to be true, another signals it).
- Functions: `pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)`, `pthread_cond_signal(pthread_cond_t *cond)`.
- Wait must always be used with a lock — it releases the lock while sleeping, and reacquires it when woken.
- Always check conditions in a while loop to avoid spurious wakeups.
- Never replace condition variables with simple flags; busy-waiting wastes CPU and leads to race conditions.

Compiling and Running Threads

- Include `pthread.h` and link with the pthread library using the `-pthread` flag:
`gcc -o main main.c -Wall -pthread`

Thread Programming Guidelines

- Keep synchronization simple; avoid complex interactions between threads.
- Minimize shared data; prefer local variables when possible.
- Always initialize locks and condition variables.
- Always check return codes.
- Avoid passing stack-allocated data to threads.
- Each thread has its own stack; share via heap/global memory only.
- Always use condition variables for signaling, not flags.
- Refer to pthread man pages for detailed documentation.

Summary

The pthread library provides a powerful interface for concurrent programming. It includes thread creation (`pthread_create`), completion (`pthread_join`), synchronization (mutexes), and signaling (condition variables). Correct use requires proper initialization, error checking, and disciplined synchronization to avoid races, deadlocks, and inefficiency.